

# Decisions for Software Design

## and beyond...



Jacqui Read

Chief Architect  
@ The Ministry of Software Design  
Author of *Communication Patterns*



[JacquiRead.com](http://JacquiRead.com)



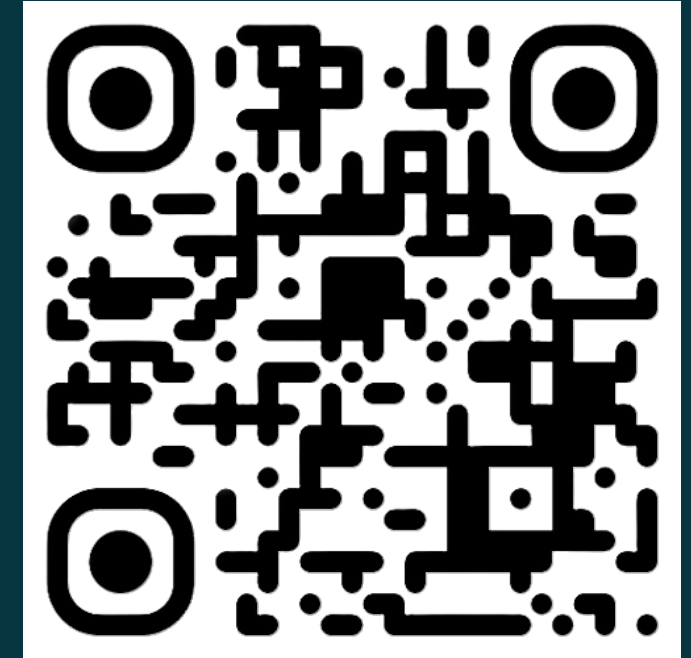
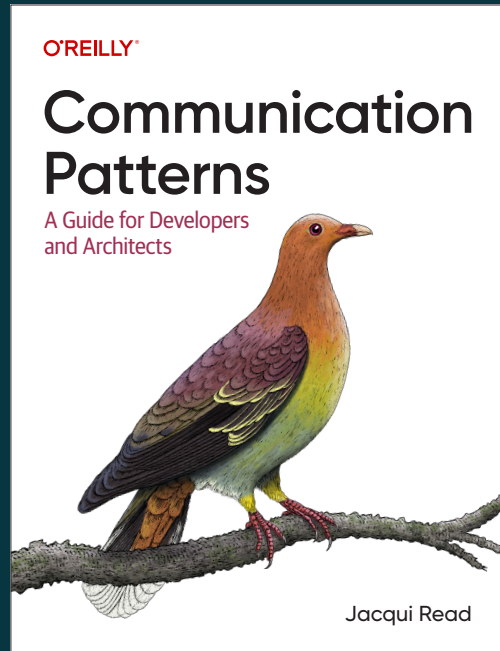
[linkedin.com/in/jacquelineread](https://linkedin.com/in/jacquelineread)



[fosstodon.org/@tekiegirl](https://fosstodon.org/@tekiegirl)



[tekiegirl.bsky.social](https://tekiegirl.bsky.social)



Jacqui Read

Chief Architect  
@ The Ministry of Software Design  
Author of *Communication Patterns*



[acedmodel.com](http://acedmodel.com)



[JacquiRead.com](http://JacquiRead.com)



[linkedin.com/in/jacquelineread](https://linkedin.com/in/jacquelineread)



[fosstodon.org/@tekiegirl](mailto:fosstodon.org/@tekiegirl)



[tekiegirl.bsky.social](https://tekiegirl.bsky.social)

There is no decision that we can  
make that doesn't come with  
some sort of balance or sacrifice.

Simon Sinek

# Overview

what

why

which

how

who

types

where

AI

myths

# What is an Architecture Decision Record?

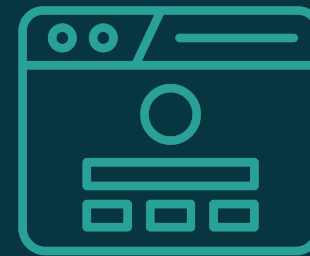
an overview and short history

The decision, the reasoning,  
and the consequences

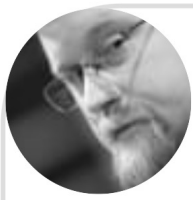
# Lightweight record or Enabling the decision process



wiki



KM



# DOCUMENTING ARCHITECTURE DECISIONS

[Michael Nygard](#) - November 15, 2011

AGILITY

ARCHITECTURE

## CONTEXT

Architecture for agile projects has to be described and defined differently. Not all decisions will be made at once, nor will all of them be done when the project begins.

Agile methods are not opposed to documentation, only to valueless documentation. Documents that assist the team itself can have value, but only if they are kept up to date. Large documents are never kept up to date. Small, modular documents have at least a chance at being updated.

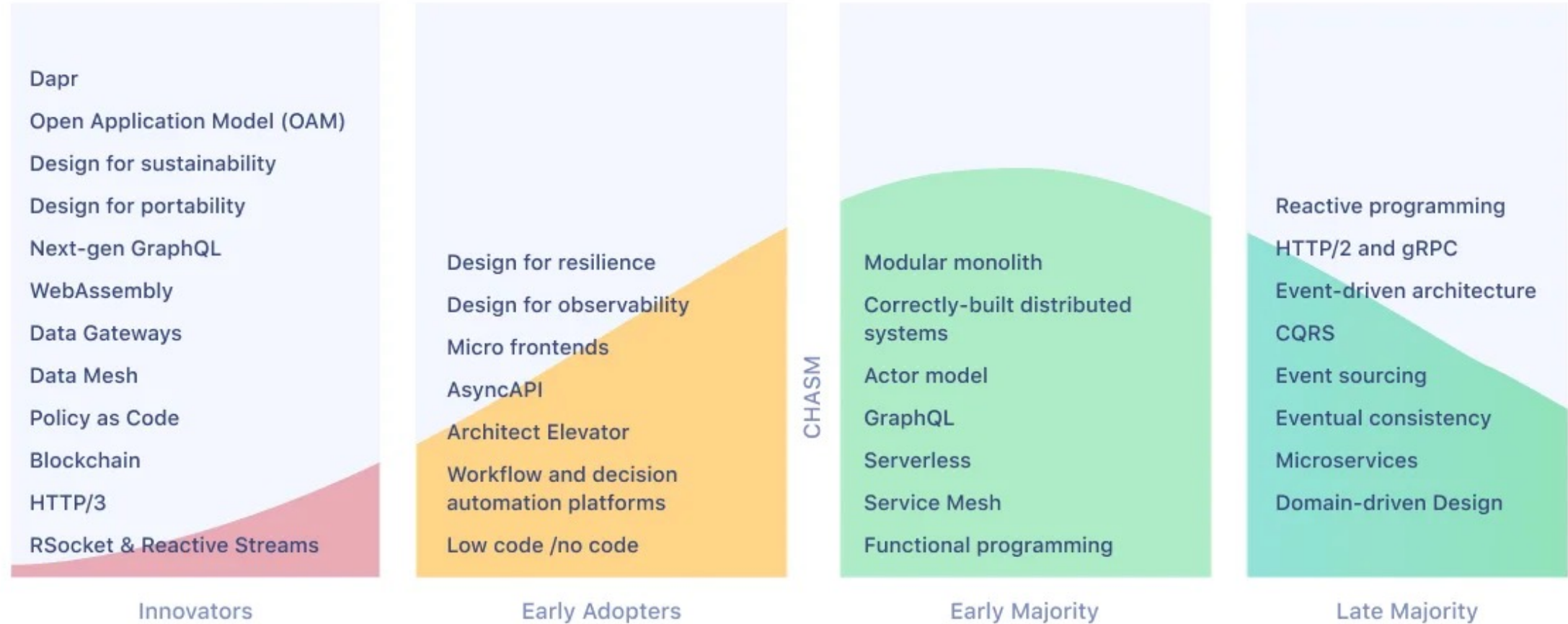
Nobody ever reads large documents, either. Most developers have been on at least one project where the specification document was larger (in bytes) than the total source code size. Those documents are too large to open, read, or update. Bite sized pieces are easier for all stakeholders to consume.

One of the hardest things to track during the life of a project is the motivation behind certain decisions. A new person coming on to a project may be perplexed, baffled, delighted, or infuriated by some past decision. Without understanding the rationale or consequences, this person has only two choices:

## Software Development Architecture and Design 2021 Graph

<http://infoq.link/architecture-trends-2021>

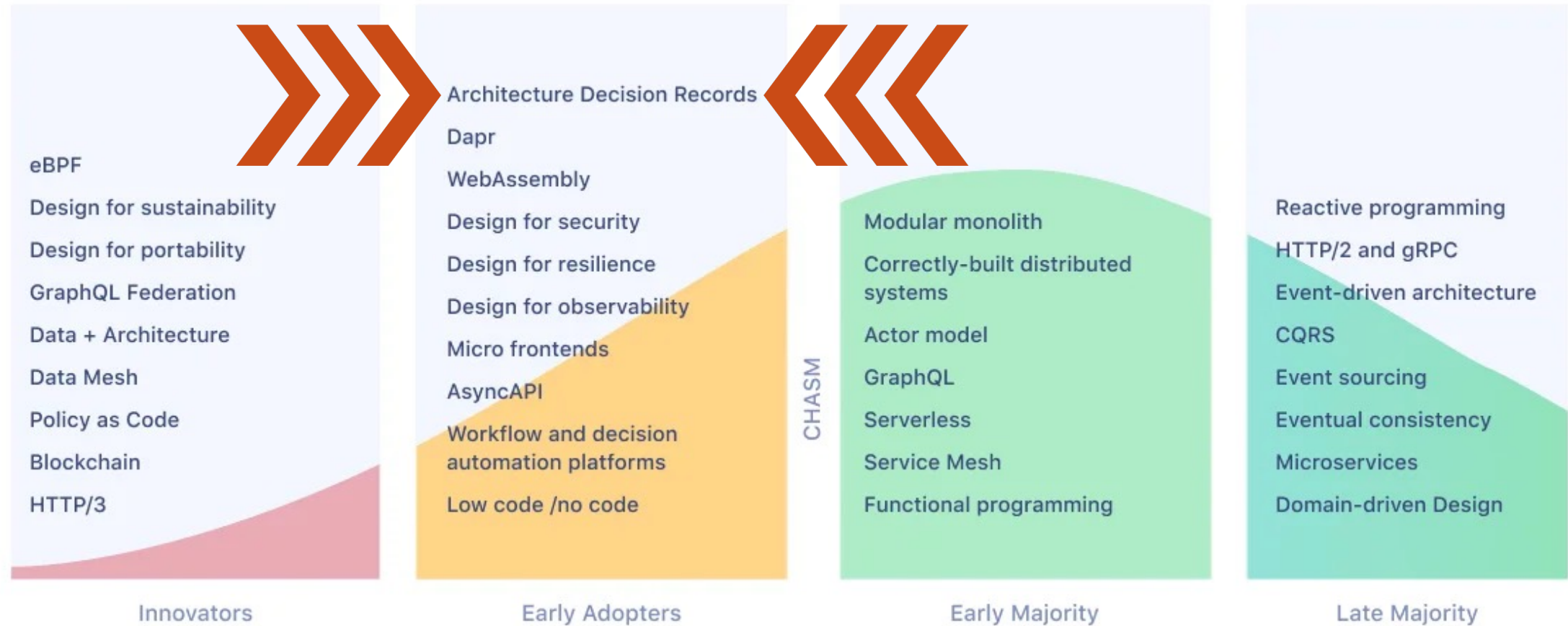
InfoQ



## Software Development Architecture and Design 2022 Graph

<http://infoq.link/architecture-trends-2022>

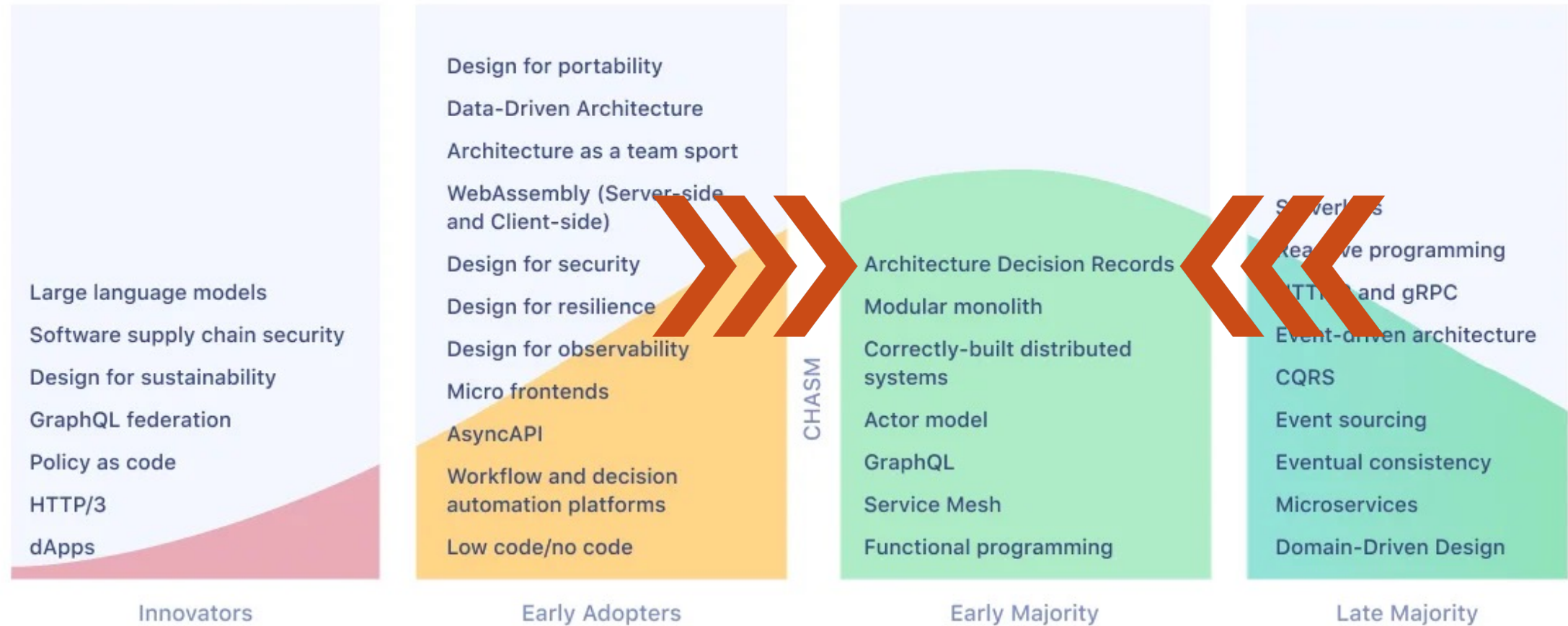
InfoQ



## Software Development Architecture and Design 2023 Graph

<http://infoq.link/architecture-trends-2023>

InfoQ

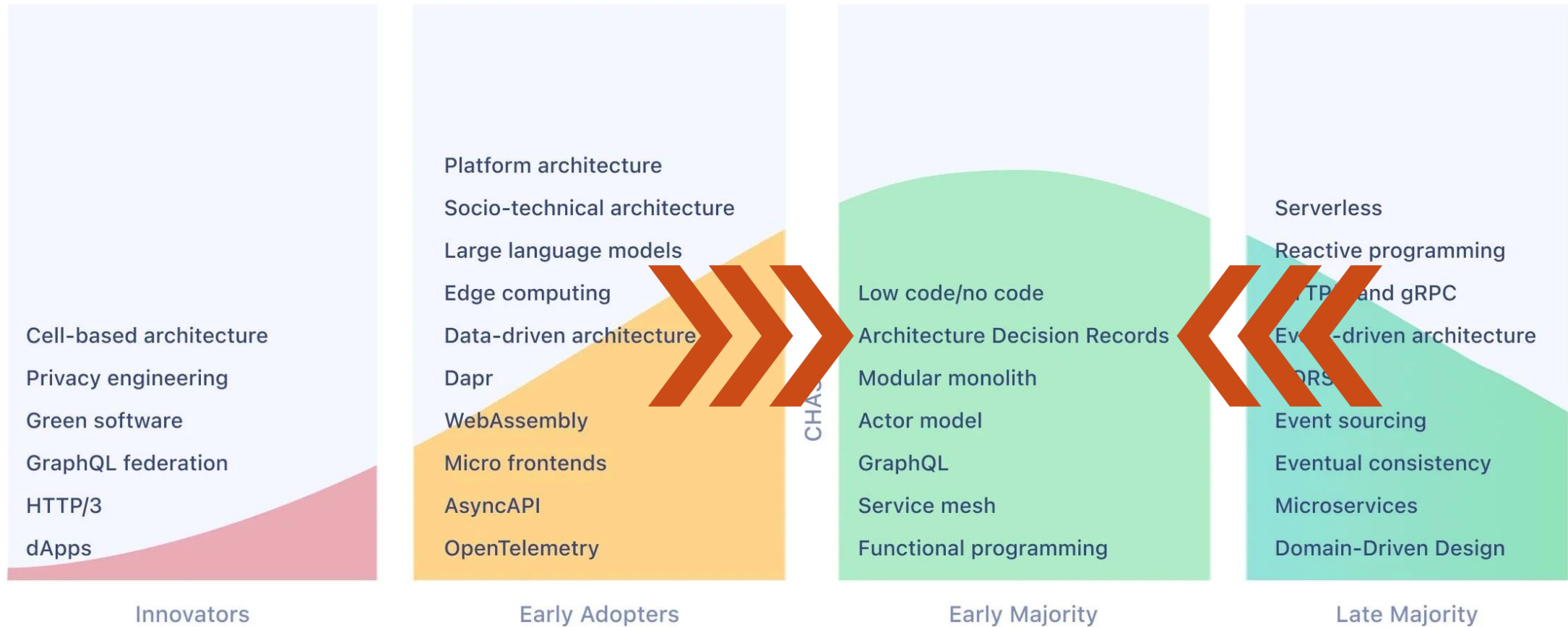


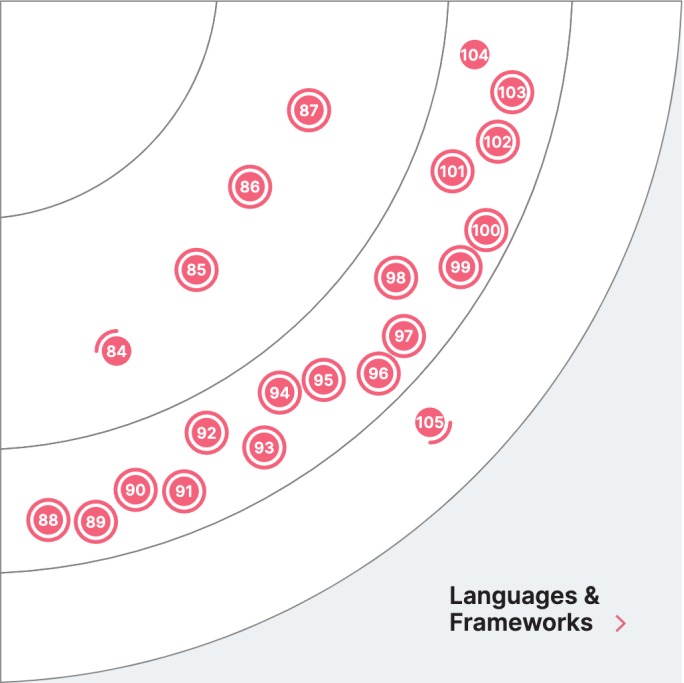
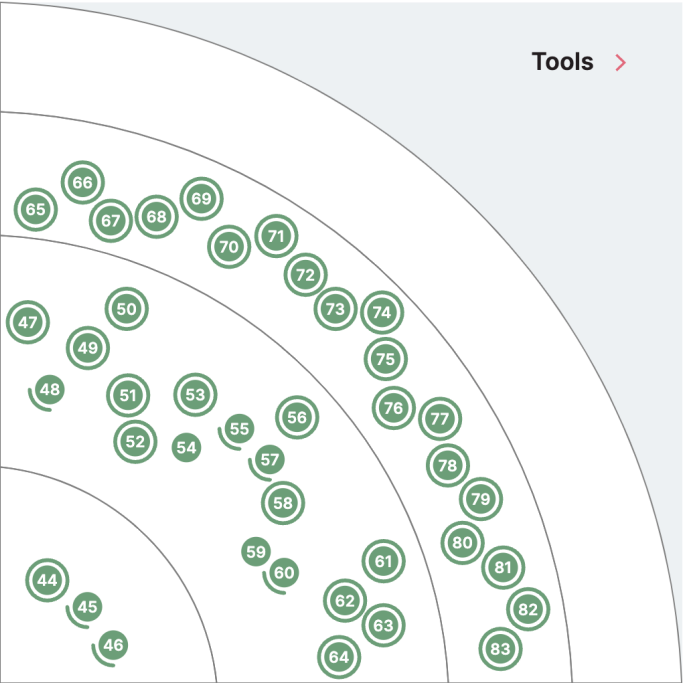
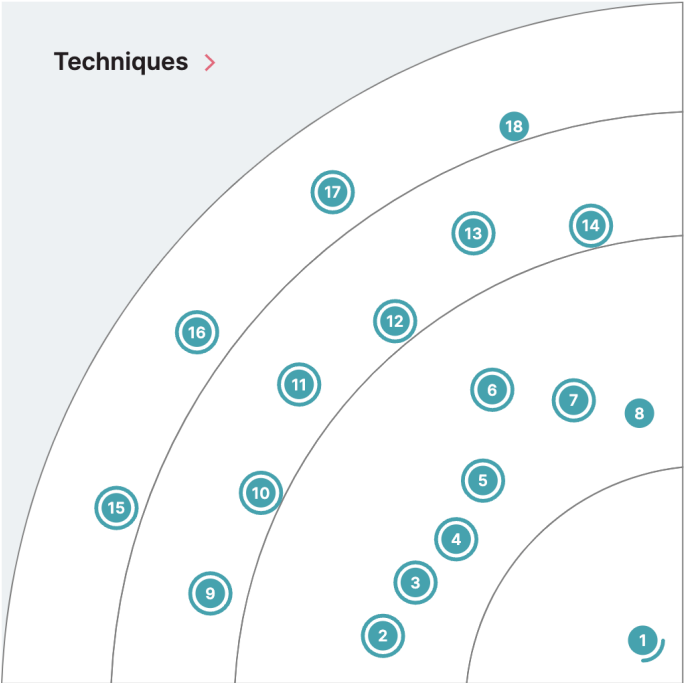


## Software Architecture and Design Trends Report – 2024

<https://www.infoq.com/articles/architecture-trends-2024/>

Read full article on  
InfoQ.com







# Lightweight Architecture Decision Records

MAY  
2018

NOV  
2017

MAR  
2017

NOV  
2016

## Trial ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control ↗ with simple markup.

MAY  
2018NOV  
2017MAR  
2017NOV  
2016The logo for Thoughtworks, featuring a red diagonal slash followed by the word "thoughtworks" in a lowercase, sans-serif font.

# Lightweight Architecture Decision Records

## Trial ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control ↗ with simple markup.

## Trial ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control ↗ with simple markup.

MAY  
2018NOV  
2017MAR  
2017NOV  
2016The Thoughtworks logo, featuring a red diagonal slash followed by the word "thoughtworks" in a white, lowercase, sans-serif font.

# Lightweight Architecture Decision Records

## Adopt ?

Much documentation can be replaced with highly readable code and tests. In a world of evolutionary architecture, however, it's important to record certain design decisions for the benefit of future team members as well as for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. We recommend storing these details in source control ↗, instead of a wiki or website, as then they can provide a record that remains in sync with the code itself. For most projects, we see no reason why you wouldn't want to use this technique.

## Trial ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control ↗ with simple markup.

## Trial ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control ↗ with simple markup.

MAY  
2018**Adopt** ?

Much documentation can be replaced with highly readable code and tests. In a world of evolutionary architecture, however, it's important to record certain design decisions for the benefit of future team members as well as for external oversight. **Lightweight Architecture Decision Records** is a technique [↗](#) for capturing important architectural decisions along with their context and consequences. We recommend storing these details in source control [↗](#), instead of a wiki or website, as then they can provide a record that remains in sync with the code itself. For most projects, we see no reason why you wouldn't want to use this technique.

NOV  
2017**Adopt** ?

Much documentation can be replaced with highly readable code and tests. In a world of evolutionary architecture, however, it's important to record certain design decisions for the benefit of future team members as well as for external oversight. **Lightweight Architecture Decision Records** is a technique [↗](#) for capturing important architectural decisions along with their context and consequences. We recommend storing these details in source control [↗](#), instead of a wiki or website, as then they can provide a record that remains in sync with the code itself. For most projects, we see no reason why you wouldn't want to use this technique.

MAR  
2017**Trial** ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique [↗](#) for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control [↗](#) with simple markup.

NOV  
2016**Trial** ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique [↗](#) for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control [↗](#) with simple markup.

The Thoughtworks logo, featuring a red diagonal slash followed by the word "thoughtworks" in a white, lowercase, sans-serif font.

# Lightweight Architecture Decision Records

MAY  
2018**Adopt** ?

Much documentation can be replaced with highly readable code and tests. In a world of evolutionary

For most projects, we see no reason why you  
wouldn't want to use this technique.

NOV  
2017**Adopt** ?

Much documentation can be replaced with highly readable code and tests. In a world of evolutionary architecture, however, it's important to record certain design decisions for the benefit of future team members as well as for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. We recommend storing these details in source control ↗, instead of a wiki or website, as then they can provide a record that remains in sync with the code itself. For most projects, we see no reason why you wouldn't want to use this technique.

MAR  
2017**Trial** ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control ↗ with simple markup.

NOV  
2016**Trial** ?

Although much documentation can be replaced with highly readable code and tests, in a world of evolutionary architecture it's important to record certain design decisions for the benefit of future team members and for external oversight. **Lightweight Architecture Decision Records** is a technique ↗ for capturing important architectural decisions along with their context and consequences. Although these items are often stored in a wiki or collaboration tool, we generally prefer storing them in source control ↗ with simple markup.

The Thoughtworks logo, featuring a red diagonal slash followed by the word "thoughtworks" in a lowercase, sans-serif font.

# Lightweight Architecture Decision Records

# Why use ADRs?

benefits and avoiding the pain

# Avoid problems with

Changing important  
decisions

Whack-a-mole  
decisions

Onboarding

Staff turnover  
& memory

# ADR Benefits

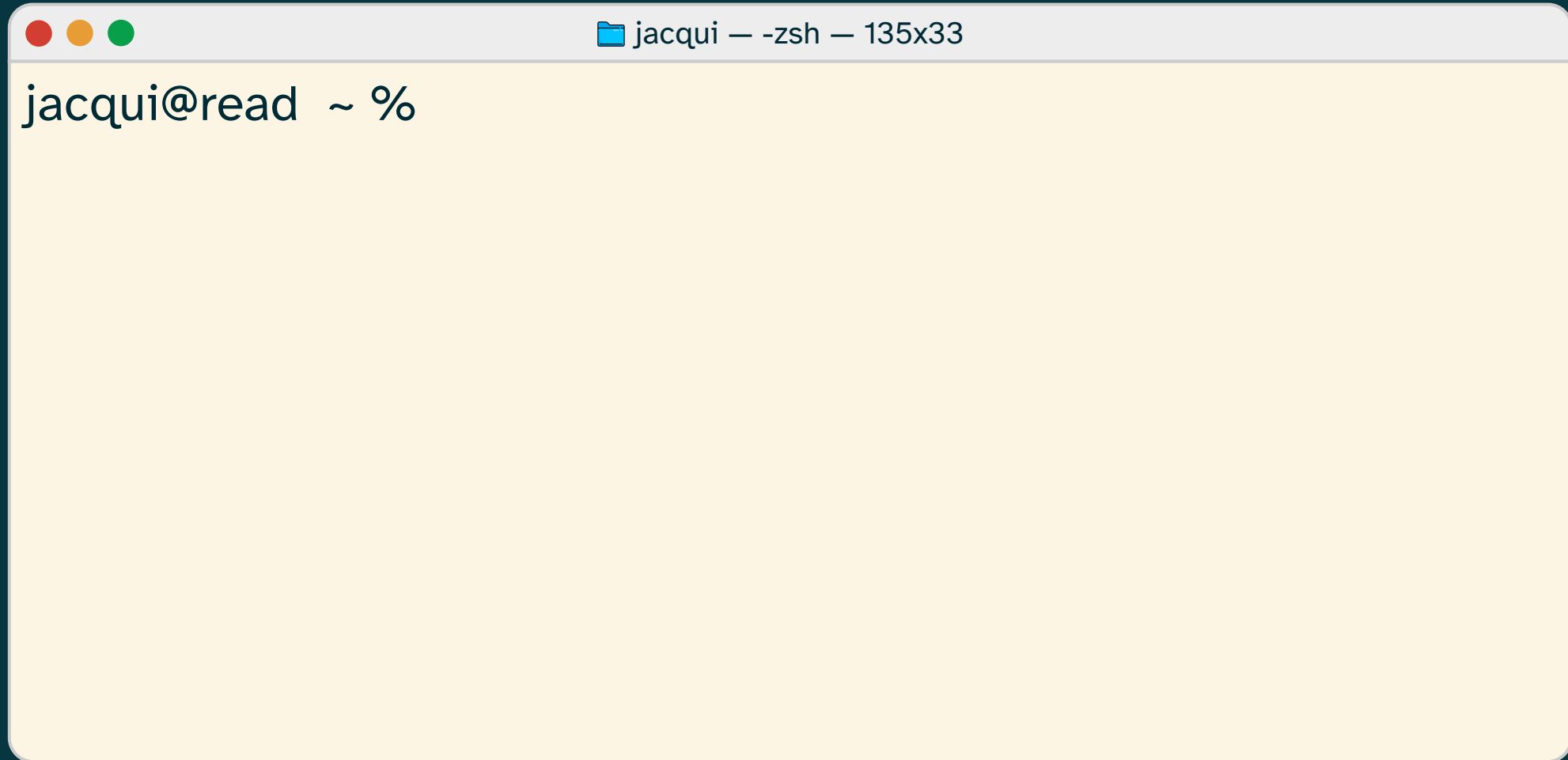
Determine if a decision  
needs review

Align with and learn from  
others

Apply the same reasoning to  
similar but new choices

(help) AI  
do the thing right

# Why do we need to remember?



# Finish the story



jacqui — -zsh — 135x33

The story ends! Sort of.

Your architecture never really ends either. Things always change.

Remembering which choices you made was important to avoiding the same problems.

But it didn't guarantee you wouldn't meet more problems.

Remembering *\*why\** you made those decisions is also important...

# Which Decisions are Architectural?

deciding which decision to record

The decision will impact the  
way developers write the  
software

The decision will be hard or  
expensive to change

# The decision is questioned

You are adopting a decision  
made by another team or  
company

The decision will have a long-lasting impact and/or affect multiple components or systems

The decision will have an effect  
outside of your team

The decision is complex or hard  
to understand

# Structuring an ADR

what to record

A problem well stated is a  
problem half-solved.

Charles Kettering

ADR-044 Use an Event-Driven Distributed Architecture

Status

Decided, 2023-10-04

Supersedes [ADR-031 Use serverless functions]

Context

The Polyglot Media system is currently a distributed system consisting mostly of serverless functions. Moving from a monolith to a distributed serverless architecture was done to tackle problems with the responsiveness of the live system and a long lead time between functionality and bug fixes being coded and deployed to production.

Serverless architecture has not solved the problems with responsiveness or maintainability to the degree required, with functions being tightly coupled with many other functions.

A solution to the problems of responsiveness and time-to-market must be found.

Evaluation Criteria

See [ADR-002 Select Architecture Characteristics]

• *Responsiveness*: Customers have been complaining about the responsiveness of the system, and this must be addressed. This is deemed the most important criterion.

• *Maintainability*: The serverless functions improved the time-to-market of bug fixes and new functionality but not to the extent needed.

• *Deployability*: Alongside maintainability, deployability is important to improve time-to-market for bug fixes and new functionality.

• *Scalability*: Most complaints from customers about responsiveness have been around peak use times, so the system must handle peak numbers of users without impacting users.

Options

1. Microservices

| Criteria        | Score     | Rationale                                                                                                                                                                                    |
|-----------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsiveness  | ★★★★☆ 3/5 | [Not inherently performant] but optimizations can be put in place at bottlenecks, e.g. scaling                                                                                               |
| Maintainability | ★★★★☆ 3/5 | Dependencies can be an issue, many data stores to maintain                                                                                                                                   |
| Deployability   | ★★★★★ 5/5 | Deploy only what has changed                                                                                                                                                                 |
| Scalability     | ★★★★★ 5/5 | Scale on per-service basis                                                                                                                                                                   |
| Total: 16/20    |           | <b>Other Trade-offs</b> <div>- Must split data into [one data store per service]</div> <div>- Costs are usually high for building</div> <div>- Complex and is hard to create workflows</div> |

2. Service-based

| Criteria        | Score     | Rationale                                                                                           |
|-----------------|-----------|-----------------------------------------------------------------------------------------------------|
| Responsiveness  | ★★★★☆ 3/5 | [Not inherently performant] but optimizations can be put in place at bottlenecks, e.g. scaling      |
| Maintainability | ★★★★☆ 4/5 | Fewer data stores to maintain than microservices                                                    |
| Deployability   | ★★★★☆ 4/5 | Deploy only what has changed, make sure changes to shared data store(s) don't affect other services |

Scalability

★★★★☆ 3/5

Scale individual services, harder to scale shared data store(s) unless using [caching]

Total: 14/20

Other Trade-offs

- Reasonably complex & hard to create workflows

- Not as evolvable as microservices

3. Event-driven

| Criteria        | Score     | Rationale                                                                                                                    |
|-----------------|-----------|------------------------------------------------------------------------------------------------------------------------------|
| Responsiveness  | ★★★★★ 5/5 | Generally [fire-and-forget], event processing can be scaled/optimized                                                        |
| Maintainability | ★★★★☆ 4/5 | Must manage event processing as well as services and data stores, events decouple services to be managed by individual teams |
| Deployability   | ★★★★☆ 3/5 | Manage event processing deployment and changes as well as services                                                           |
| Scalability     | ★★★★★ 5/5 | Services and message processing can both be scaled                                                                           |
| Total: 17/20    |           | <b>Other Trade-offs</b> <div>- [Integration testing can be harder]</div>                                                     |

Decision

We will use event-driven architecture, based on the total score against the decision criteria and the fact that options 1 and 2 did not score well on the most important criterion—responsiveness.

Implications

Positive

• We gain overall higher responsiveness, maintainability, and scalability than the current serverless functions.

• Processing of events is scalable by spinning up additional instances of services to process queued events.

• Processing within services is scalable by spinning up additional instances of services to handle the need for processing within the services.

Negative

• Teams or individuals may need to learn new skills or technologies (such as event queues).

• Event management (such as queues) adds extra complexity to development and deployment.

• Integration testing and DevOps will need to be overhauled.

Consultation

• *Vlad*: Neither microservices nor service-based are [inherently performant], so likely will not boost responsiveness as much as we want.

• *Nikki*: I have experience in event-driven architecture, using queues. We monitored the length of queues and spun up extra instances of services to process long queues when needed.

• *Libby*: Queues and channels can be used to prioritize the processing of events, for example with the [ambulance pattern].

• *Mark*: Responsiveness needs to be the top priority due to customer complaints.

• *Libby*: Pure microservices would require that services do not share data stores. Splitting our data like this would require a lot of effort.

© The Ministry of Software Design Ltd.

# ADR-044 Use an Event-Driven Distributed Architecture

## Status

Decided, 2023-10-04

Supersedes [[ADR-031 Use serverless functions](#)]

## Context

The Polyglot Media system is currently a distributed system consisting mostly of serverless functions. Moving from a monolith to a distributed serverless architecture was done to tackle problems with the responsiveness of the live system and a long lead time between functionality and bug fixes being coded and deployed to production.

Serverless architecture has not solved the problems with responsiveness or maintainability to the degree required, with functions being tightly coupled with many other functions.

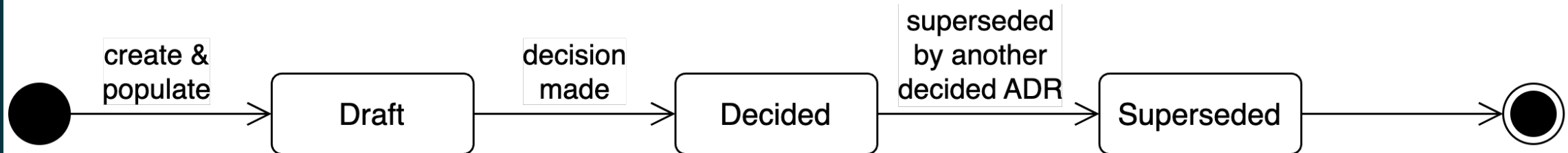
A solution to the problems of responsiveness and time-to-market must be found.

# ADR-044 Use an Event-Driven Distributed Architecture

## Status

Decided, 2023-10-04

Supersedes [[ADR-031 Use serverless functions](#)]



maintainability to the degree required, with functions being tightly coupled with many other functions.

A solution to the problems of responsiveness and time-to-market must be found.

## Evaluation Criteria

See [[ADR-002 Select Architecture Characteristics](#)]

- *Responsiveness*: Customers have been complaining about the responsiveness of the system, and this must be addressed. This is deemed the most important criterion.
- *Maintainability*: The serverless functions improved the time-to-market of bug fixes and new functionality but not to the extent needed.
- *Deployability*: Alongside maintainability, deployability is important to improve time-to-market for bug fixes and new functionality.
- *Scalability*: Most complaints from customers about responsiveness have been around peak use times, so the system must handle peak numbers of users without impacting users.

# Options

## 1. Microservices

| Criteria            | Score     | Rationale                                                                                                                                                                                                                   |
|---------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsiveness      | ★★★☆☆ 3/5 | [Not inherently performant] but optimizations can be put in place at bottlenecks, e.g. scaling                                                                                                                              |
| Maintainability     | ★★★☆☆ 3/5 | Dependencies can be an issue, many data stores to maintain                                                                                                                                                                  |
| Deployability       | ★★★★★ 5/5 | Deploy only what has changed                                                                                                                                                                                                |
| Scalability         | ★★★★★ 5/5 | Scale on per-service basis                                                                                                                                                                                                  |
| <b>Total: 16/20</b> |           | <b>Other Trade-offs</b> <ul style="list-style-type: none"><li>- Must split data into [one data store per service]</li><li>- Costs are usually high for building</li><li>- Complex and is hard to create workflows</li></ul> |

## 2. Service-based

## 2. Service-based

| Criteria            | Score     | Rationale                                                                                                                                                               |
|---------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsiveness      | ★★★☆☆ 3/5 | [Not inherently performant] but optimizations can be put in place at bottlenecks, e.g. scaling                                                                          |
| Maintainability     | ★★★★☆ 4/5 | Fewer data stores to maintain than microservices                                                                                                                        |
| Deployability       | ★★★★☆ 4/5 | Deploy only what has changed, make sure changes to shared data store(s) don't affect other services                                                                     |
| Scalability         | ★★★☆☆ 3/5 | Scale individual services, harder to scale shared data store(s) unless using [caching]                                                                                  |
| <b>Total: 14/20</b> |           | <b>Other Trade-offs</b> <ul style="list-style-type: none"><li>- Reasonably complex &amp; hard to create workflows</li><li>- Not as evolvable as microservices</li></ul> |

## 3. Event-driven

- Not as evolvable as microservices

### 3. Event-driven

| Criteria            | Score     | Rationale                                                                                                                    |
|---------------------|-----------|------------------------------------------------------------------------------------------------------------------------------|
| Responsiveness      | ★★★★★ 5/5 | Generally [ <a href="#">fire-and-forget</a> ], event processing can be scaled/optimized                                      |
| Maintainability     | ★★★★☆ 4/5 | Must manage event processing as well as services and data stores, events decouple services to be managed by individual teams |
| Deployability       | ★★★☆☆ 3/5 | Manage event processing deployment and changes as well as services                                                           |
| Scalability         | ★★★★★ 5/5 | Services and message processing can both be scaled                                                                           |
| <b>Total: 17/20</b> |           | <b>Other Trade-offs</b><br>- [ <a href="#">Integration testing can be harder</a> ]                                           |

## Decision

We will use event-driven architecture, based on the total score against the decision criteria and the fact that options 1 and 2 did not score well on the most important criterion—responsiveness.

## Implications

### *Positive*

- We gain overall higher responsiveness, maintainability, and scalability than the current serverless functions.
- Processing of events is scalable by spinning up additional instances of services to process queued events.
- Processing within services is scalable by spinning up additional instances of services to handle the need for processing within the services.

### *Negative*

- Teams or individuals may need to learn new skills or technologies (such as event queues).

important criterion: responsiveness.

## Implications

### *Positive*

- We gain overall higher responsiveness, maintainability, and scalability than the current serverless functions.
- Processing of events is scalable by spinning up additional instances of services to process queued events.
- Processing within services is scalable by spinning up additional instances of services to handle the need for processing within the services.

### *Negative*

- Teams or individuals may need to learn new skills or technologies (such as event queues).
- Event management (such as queues) adds extra complexity to development and deployment.
- Integration testing and DevOps will need to be overhauled.

## Consultation

- *Vlad*: Neither microservices nor service-based are [[inherently performant](#)], so likely will not boost responsiveness as much as we want.
- *Nikki*: I have experience in event-driven architecture, using queues. We monitored the length of queues and spun up extra instances of services to process long queues when needed.
- *Libby*: Queues and channels can be used to prioritize the processing of events, for example with the [[ambulance pattern](#)].
- *Mark*: Responsiveness needs to be the top priority due to customer complaints.
- *Libby*: Pure microservices would require that services do not share data stores. Splitting our data like this would require a lot of effort.

# ADR-044 Change to Event-Driven Architecture

## Status

Decided, 2023-10-04

Supersedes [[ADR-031 Use serverless functions](#)]

## Context

The Polyglot Media system is currently a distributed system consisting mostly of serverless functions. Moving from a monolith to a distributed serverless architecture was done to tackle problems with the responsiveness of the live system and a long lead time between functionality and bug fixes being coded and deployed to production.

Serverless architecture has not solved the problems with responsiveness or maintainability to the degree required, with functions being tightly coupled with many other functions.

A solution to the problems of responsiveness and time-to-market must be found.

# Decision

We will use Event-Driven architecture, based on its high scalability and responsiveness. Microservices and service-based don't support the most important criterion—responsiveness.

## Implications

### *Positive*

- We gain overall higher responsiveness, maintainability, and scalability than the current serverless functions.
- Processing of events is scalable by spinning up additional instances of services to process queued events.
- Processing within services is scalable by spinning up additional instances of services to handle the need for processing within the services.

### *Negative*

- Teams or individuals may need to learn new skills or technologies (such as

# Implications

## *Positive*

- We gain overall higher responsiveness, maintainability, and scalability than the current serverless functions.
- Processing of events is scalable by spinning up additional instances of services to process queued events.
- Processing within services is scalable by spinning up additional instances of services to handle the need for processing within the services.

## *Negative*

- Teams or individuals may need to learn new skills or technologies (such as event queues).
- Event management (such as queues) adds extra complexity to development and deployment.
- Integration testing and DevOps will need to be overhauled.

# Examples

ADR-044 Use an Event-Driven Distributed Architecture

Status

Decided, 2023-10-04  
Supersedes [ADR-031 Use serverless functions]

Context

The Polyglot Media system is currently a distributed system consisting mostly of serverless functions. Moving from a monolith to a distributed serverless architecture was done to tackle problems with the responsiveness of the live system and a long lead time between functionality and bug fixes being coded and deployed to production.  
  
Serverless architecture has not solved the problems with responsiveness or maintainability to the degree required, with functions being tightly coupled with many other functions.  
  
A solution to the problems of responsiveness and time-to-market must be found.

Evaluation Criteria

See [ADR-002 Select Architecture Characteristics]

- Responsiveness*: Customers have been complaining about the responsiveness of the system, and this must be addressed. This is deemed the most important criterion.
- Maintainability*: The serverless functions improved the time-to-market of bug fixes and new functionality but not to the extent needed.
- Deployability*: Alongside maintainability, deployability is important to improve time-to-market for bug fixes and new functionality.
- Scalability*: Most complaints from customers about responsiveness have been around peak use times, so the system must handle peak numbers of users without impacting users.

Options

1. Microservices

| Criteria        | Score     | Rationale                                                                                                                                                                                                                   |
|-----------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsiveness  | ★★★★☆ 3/5 | [Not inherently performant] but optimizations can be put in place at bottlenecks, e.g. scaling                                                                                                                              |
| Maintainability | ★★★★☆ 3/5 | Dependencies can be an issue, many data stores to maintain                                                                                                                                                                  |
| Deployability   | ★★★★★ 5/5 | Deploy only what has changed                                                                                                                                                                                                |
| Scalability     | ★★★★★ 5/5 | Scale on per-service basis                                                                                                                                                                                                  |
| Total: 16/20    |           | <b>Other Trade-offs</b> <ul style="list-style-type: none"><li>- Must split data into [one data store per service]</li><li>- Costs are usually high for building</li><li>- Complex and is hard to create workflows</li></ul> |

2. Service-based

| Criteria        | Score     | Rationale                                                                                           |
|-----------------|-----------|-----------------------------------------------------------------------------------------------------|
| Responsiveness  | ★★★★☆ 3/5 | [Not inherently performant] but optimizations can be put in place at bottlenecks, e.g. scaling      |
| Maintainability | ★★★★☆ 4/5 | Fewer data stores to maintain than microservices                                                    |
| Deployability   | ★★★★☆ 4/5 | Deploy only what has changed, make sure changes to shared data store(s) don't affect other services |

Scalability

★★★★☆ 3/5

Scale individual services, harder to scale shared data store(s) unless using [caching]

Total: 14/20

Other Trade-offs

- Reasonably complex & hard to create workflows
- Not as evolvable as microservices

3. Event-driven

| Criteria        | Score     | Rationale                                                                                                                    |
|-----------------|-----------|------------------------------------------------------------------------------------------------------------------------------|
| Responsiveness  | ★★★★★ 5/5 | Generally [fire-and-forget], event processing can be scaled/optimized                                                        |
| Maintainability | ★★★★☆ 4/5 | Must manage event processing as well as services and data stores, events decouple services to be managed by individual teams |
| Deployability   | ★★★★☆ 3/5 | Manage event processing deployment and changes as well as services                                                           |
| Scalability     | ★★★★★ 5/5 | Services and message processing can both be scaled                                                                           |
| Total: 17/20    |           | <b>Other Trade-offs</b> <ul style="list-style-type: none"><li>- [Integration testing can be harder]</li></ul>                |

Decision

We will use event-driven architecture, based on the total score against the decision criteria and the fact that options 1 and 2 did not score well on the most important criterion—responsiveness.

Implications

Positive

- We gain overall higher responsiveness, maintainability, and scalability than the current serverless functions.
- Processing of events is scalable by spinning up additional instances of services to process queued events.
- Processing within services is scalable by spinning up additional instances of services to handle the need for processing within the services.

Negative

- Teams or individuals may need to learn new skills or technologies (such as event queues).
- Event management (such as queues) adds extra complexity to development and deployment.
- Integration testing and DevOps will need to be overhauled.

Consultation

- Vlad*: Neither microservices nor service-based are [inherently performant], so likely will not boost responsiveness as much as we want.
- Nikki*: I have experience in event-driven architecture, using queues. We monitored the length of queues and spun up extra instances of services to process long queues when needed.
- Libby*: Queues and channels can be used to prioritize the processing of events, for example with the [ambulance pattern].
- Mark*: Responsiveness needs to be the top priority due to customer complaints.
- Libby*: Pure microservices would require that services do not share data stores. Splitting our data like this would require a lot of effort.

ADR-044 Change to Event-Driven Architecture

Status

Decided, 2023-10-04  
Supersedes [ADR-031 Use serverless functions]

Context

The Polyglot Media system is currently a distributed system consisting mostly of serverless functions. Moving from a monolith to a distributed serverless architecture was done to tackle problems with the responsiveness of the live system and a long lead time between functionality and bug fixes being coded and deployed to production.  
  
Serverless architecture has not solved the problems with responsiveness or maintainability to the degree required, with functions being tightly coupled with many other functions.  
  
A solution to the problems of responsiveness and time-to-market must be found.

Decision

We will use Event-Driven architecture, based on its high scalability and responsiveness. Microservices and service-based don't support the most important criterion—responsiveness.

Implications

Positive

- We gain overall higher responsiveness, maintainability, and scalability than the current serverless functions.
- Processing of events is scalable by spinning up additional instances of services to process queued events.
- Processing within services is scalable by spinning up additional instances of services to handle the need for processing within the services.

Negative

- Teams or individuals may need to learn new skills or technologies (such as event queues).
- Event management (such as queues) adds extra complexity to development and deployment.
- Integration testing and DevOps will need to be overhauled.

CC

i

BY

# References in ADRs

Requirements

Constraints

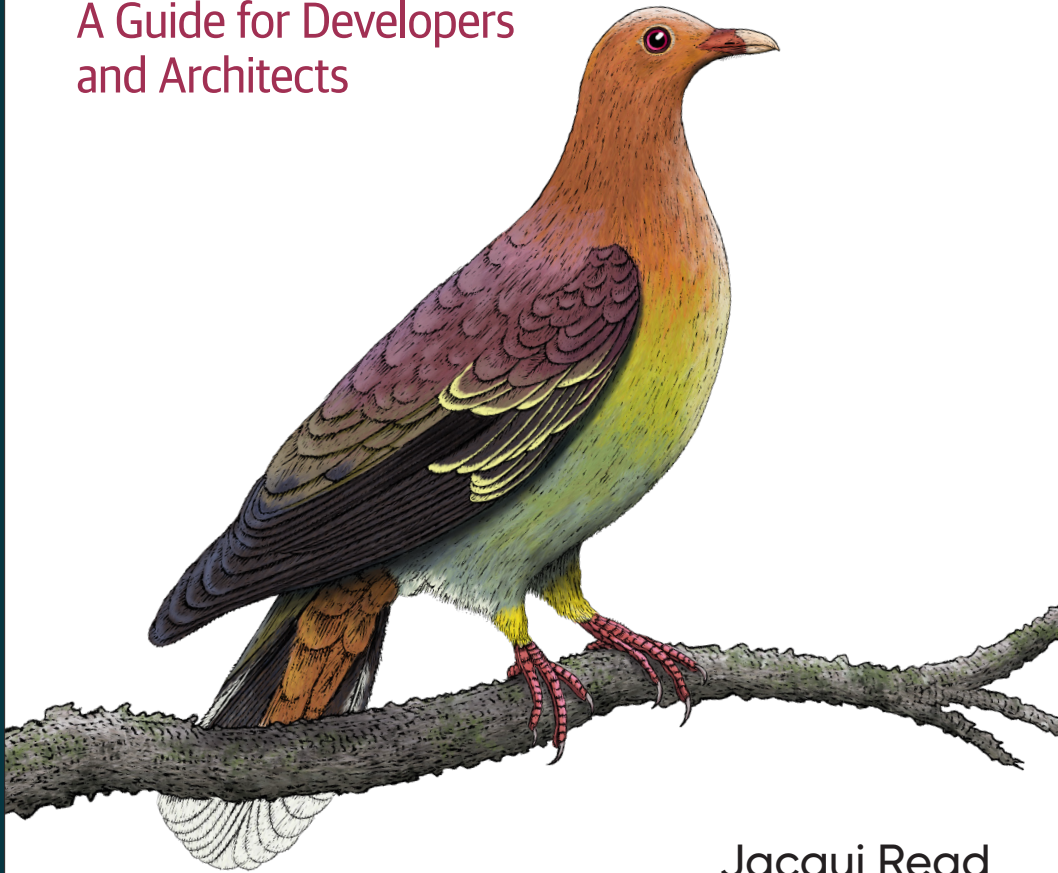
Architecture  
Characteristics

Other ADRs

O'REILLY®

# Communication Patterns

A Guide for Developers  
and Architects



Jacqui Read

Free ADR templates  
and examples

[CommunicationPatternsBook.com](http://CommunicationPatternsBook.com)

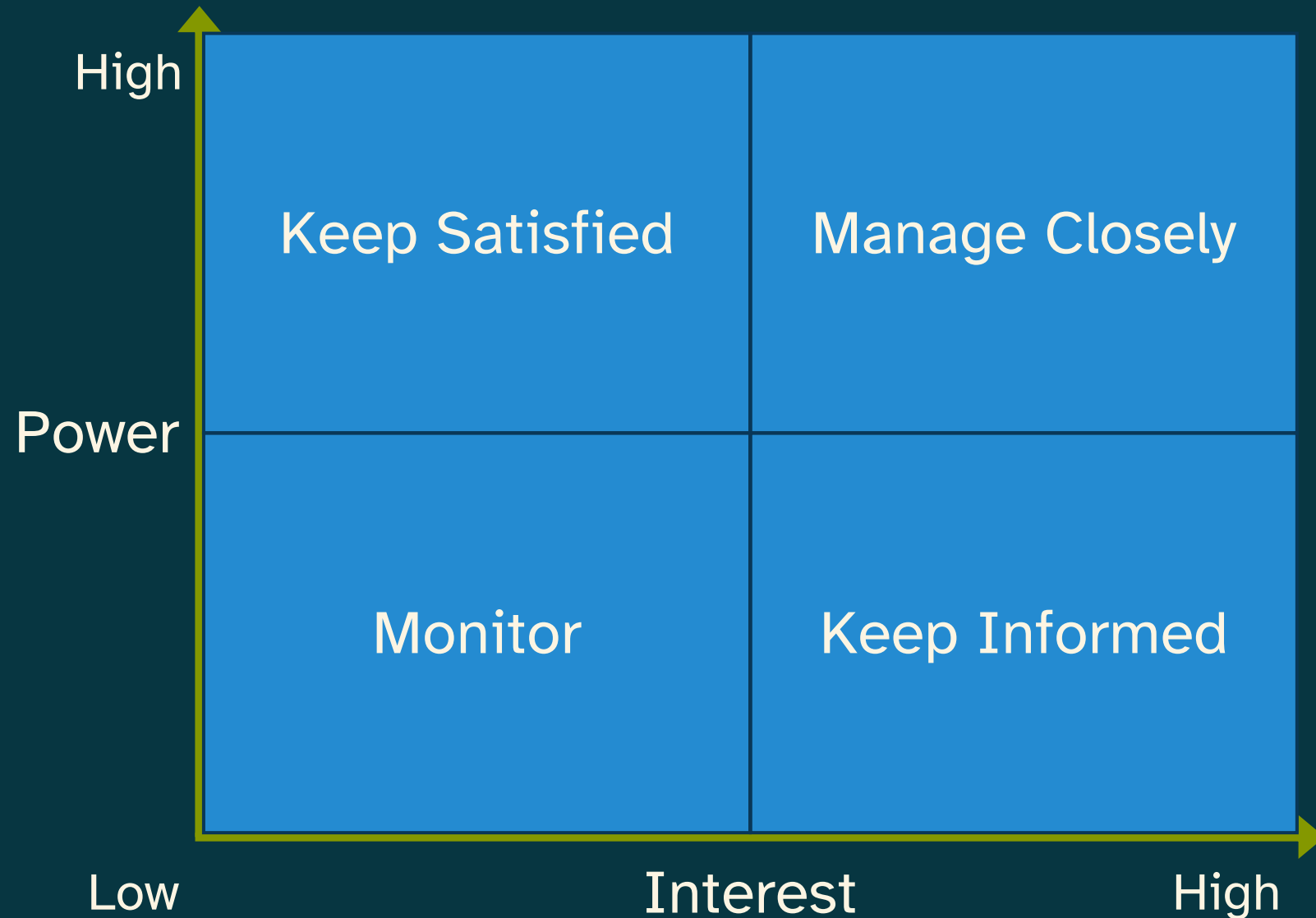


# Who is involved in ADRs?

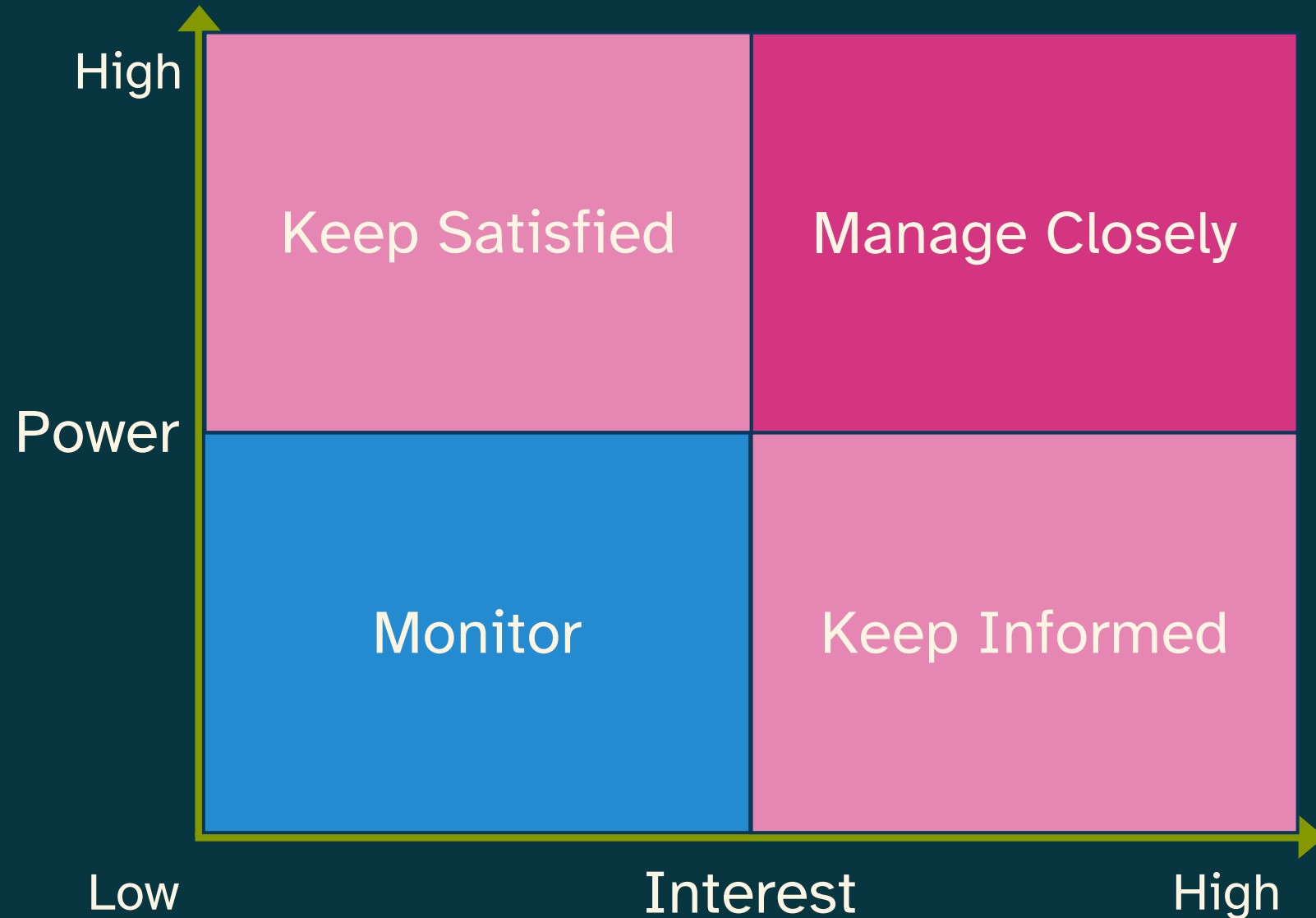
the stakeholders



# Power Interest Grid



# Power Interest Grid





# Historical ADRs

decisions in brownfield products

Should you create ADRs for  
past decisions?

# Additional Benefits for HADRs

Higher Hit Rate

Review As-You-Go

Record & Reuse  
Investigations/Discussions

# Historical ADR Tips

Create on  
Discovery

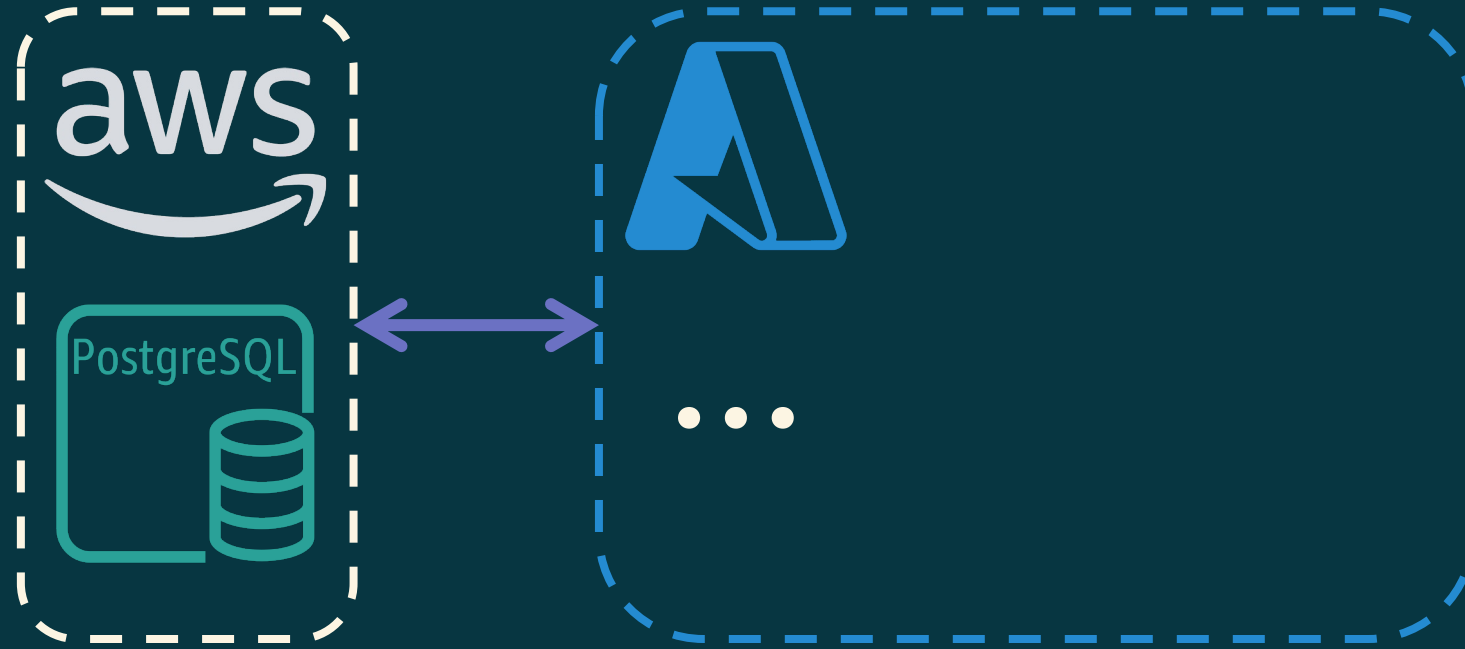
Collaborate

Use  
Documentation

Highlight  
Assumptions

Link to  
Superseded

Add  
Hindsight



move to AWS

supersedes



move to Azure



leave data in AWS

# Which ADRs take priority?



# Biased ADRs

an antipattern

An ADR that justifies  
the decision...

An ADR that justifies  
the decision...  
but ignores **inconvenient** facts



# Business Decision Records

decisions beyond software

# Business Decision Records

Procurement

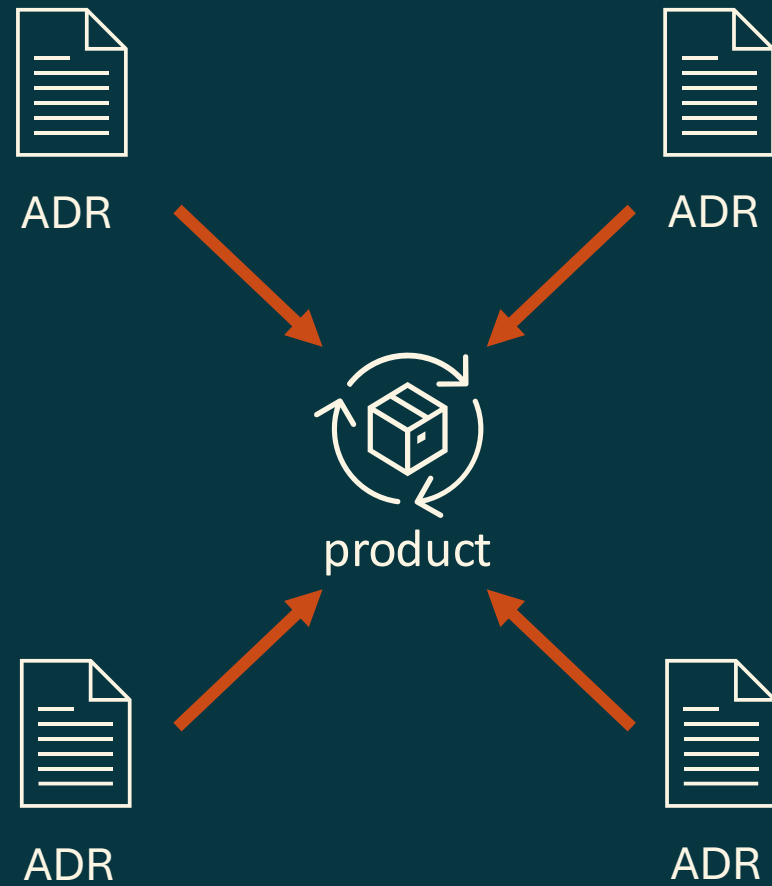
Security

Hiring

Strategy

# Adapt BDRs to the context







SDR



# Storing ADRs

and other decision records

Creating ADRs is not  
the end of the story

Store Decision Records where  
people can find and access  
them easily

# Decision Record Storage

Permitted & Easy

Fit Workflows

Code-Related in the  
Repository

Wide-Ranging  
Centralised

# Link to decisions from a RAID

# Centralise and link

# ADR Culture

further ways to get people using ADRs

show the benefits,  
rather than tell

# Practical ADRs

Answer with  
an ADR

Reference ADRs

ADRs for  
Onboarding

Discuss ADRs

Create  
Historical ADRs

Make it Easy

# Practical ADRs

**DON'T GIVE UP**

Answer with  
an ADR

Reference ADRs

ADRs for  
Onboarding

Discuss ADRs

Create  
Historical ADRs

Make it Easy

# AI and Decision Records

where artificial intelligence fits in



# AI to the rescue?

LLMs do not understand

We are the ones making the  
decisions

LLMs can help research  
historical Decision Records

LLMs can answer questions  
based on Decision Records  
as input

# Decision-Making Myths

quash these for better decision-making

# Myth 1: Decision making is linear



Myth 2:  
Giving more choices is better  
for making decisions



Myth 3:  
The decision should be made  
by the most senior person



Myth 4:  
All stakeholders should be involved  
in the feedback process



Myth 5:  
You should ask for  
all kinds of feedback

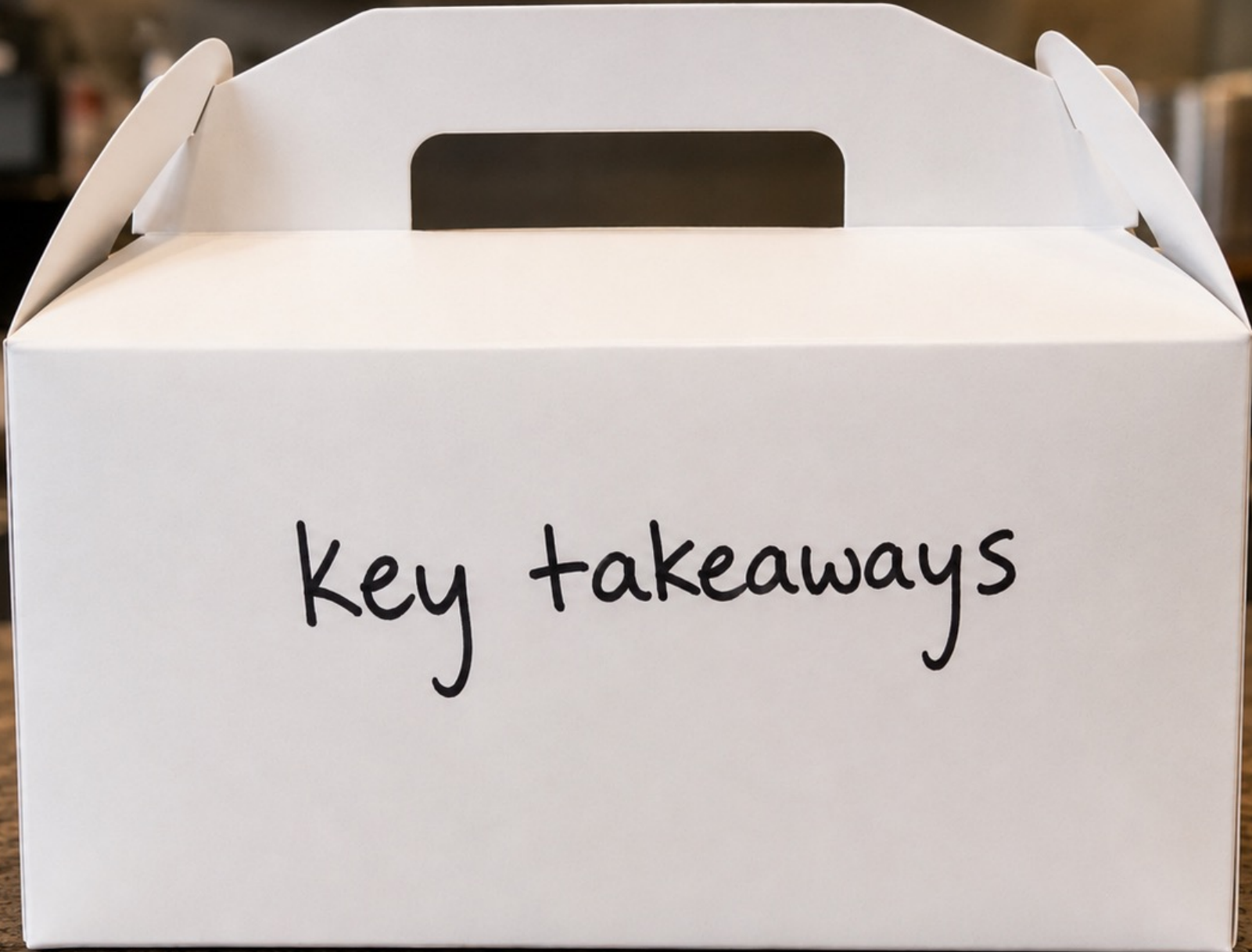


Myth 6:  
All stakeholders must  
agree on the outcome



Myth 7:  
The decision owner can  
make a rational decision





Key takeaways

# Make decisions together

Don't make decisions you  
shouldn't be making

Use ADRs so you know if it is  
safe to change a decision...

... but to also quickly see if a  
decision needs to be reviewed

# Learn from ADRs

# Create historical ADRs

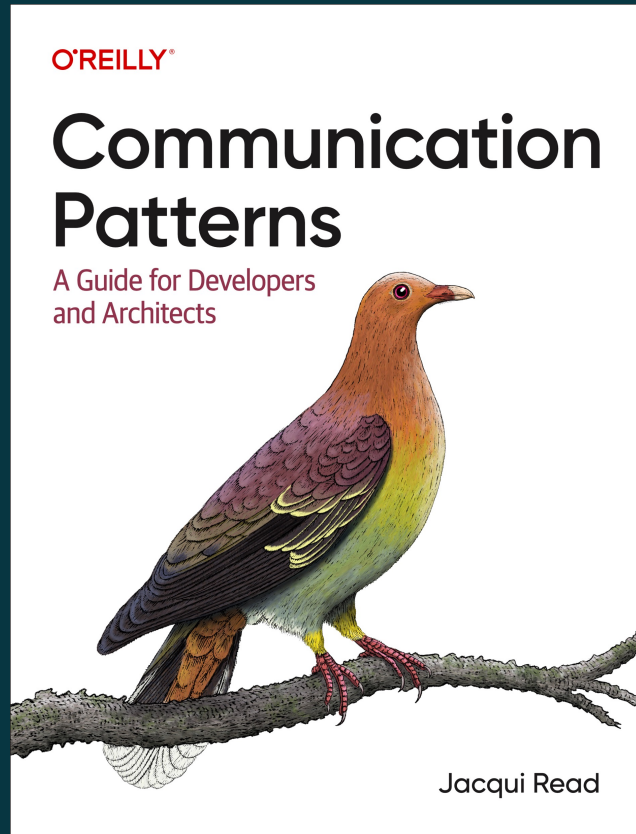
BDRs extend the benefits across  
your organisation

Make using ADRs easy and part  
of everyone's workflow

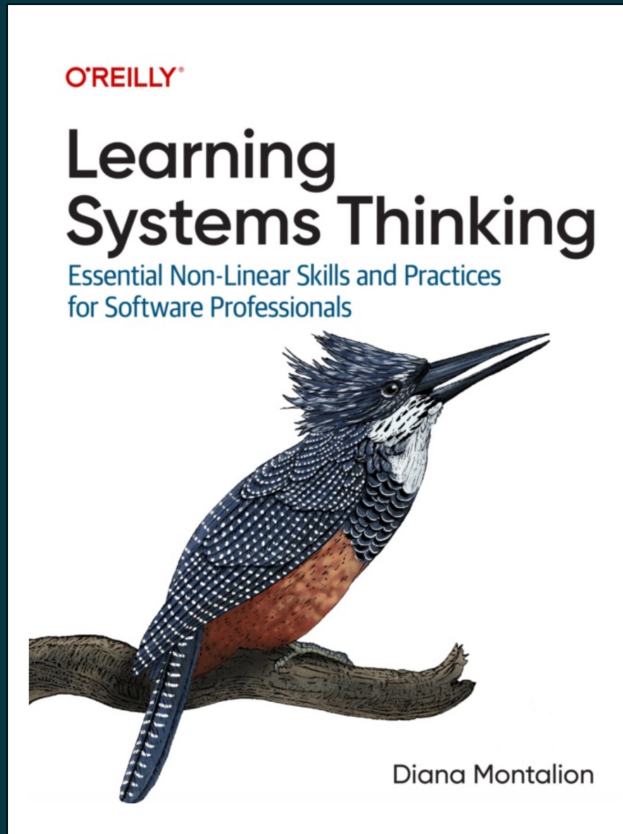
Answer with a link

No one regrets ADRs

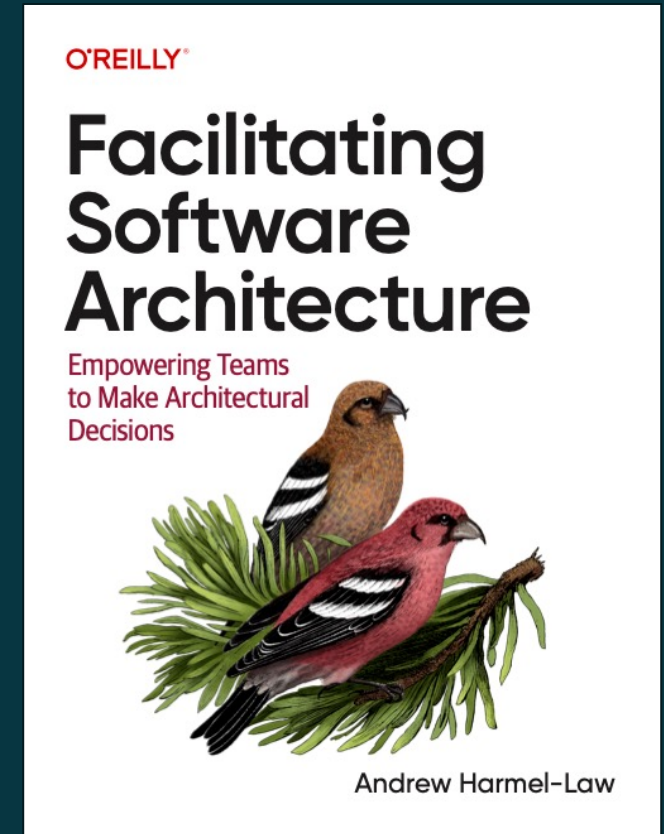
# Decision Book Recommendations



Jacquie Read

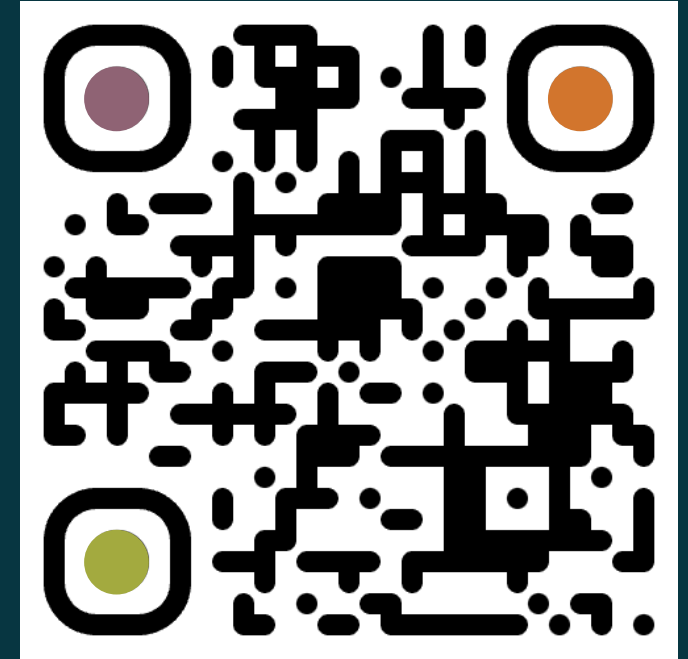
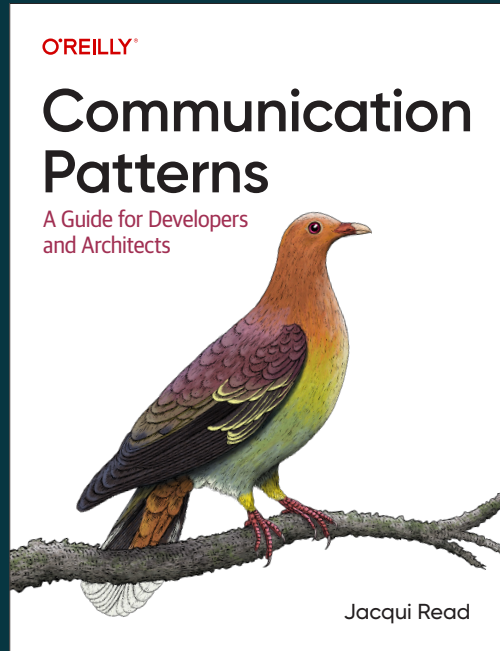


Diana Montalion



Andrew Harmel-Law

# Thank you



Jacqui Read

Chief Architect  
@ The Ministry of Software Design  
Author of *Communication Patterns*



[acedmodel.com](http://acedmodel.com)



[JacquiRead.com](http://JacquiRead.com)



[linkedin.com/in/jacquelineread](https://linkedin.com/in/jacquelineread)



[fosstodon.org/@tekiegirl](mailto:fosstodon.org/@tekiegirl)



[tekiegirl.bsky.social](https://tekiegirl.bsky.social)

# Further Reading

- <https://jacquiread.com>
- <https://communicationpatternsbook.com>
- <https://jacquiread.com/posts/2024-02-26-historical-adrs/>
- <https://learning.oreilly.com/library/view/communication-patterns/9781098140533/ch12.html>
- <https://learning.oreilly.com/library/view/facilitating-software-architecture/9781098151850/>
- <https://learning.oreilly.com/library/view/learning-systems-thinking/9781098151324/>
- <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- <https://www.projectmanagement.com/wikis/368897/Stakeholder-Analysis--using-the-Power-Interest-Grid>
- <https://www.thoughtworks.com/radar/techniques/lightweight-architecture-decision-records>
- <https://www.infoq.com/articles/architecture-trends-2021/> (2021-2024)